SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: **Artificial Intelligence** **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

sDry Run Evaluation

Code of Conduct:

I Athavan K (192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

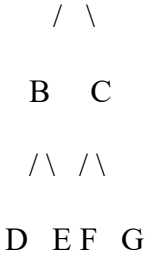
Signature of the student:Athavan K

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



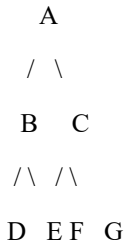
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Perform **Depth-First Search (DFS)** starting from **A**.

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Complete the table.

3. **Initial State :**

1 3 6

5 0 2

4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

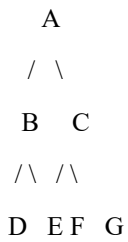
1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Question 1 — Breadth-First Search (BFS)

Given tree (rooted at A):



BFS explores the tree level by level using a FIFO queue: a node is enqueued when it is first discovered, and dequeued (visited/expanded) in the same order it was enqueued. Children are enqueued left to right (B before C, D before E, F before G).

Dry-Run Table

Iteration	Queue (before expansion)	Node Expanded / Visited	Queue (after expansion)
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	(empty)

1. BFS Traversal Order

A → B → C → D → E → F → G

2. Queue Contents After Each Iteration

- Iteration 1 (expand A): Queue = [B, C]
- Iteration 2 (expand B): Queue = [C, D, E]
- Iteration 3 (expand C): Queue = [D, E, F, G]
- Iteration 4 (expand D): Queue = [E, F, G] — D has no children, so nothing is added
- Iteration 5 (expand E): Queue = [F, G] — E has no children
- Iteration 6 (expand F): Queue = [G] — F has no children
- Iteration 7 (expand G): Queue = [ ] (empty) — G has no children; BFS terminates as the queue is exhausted

Question 2 — Depth-First Search (DFS)

Using the same tree, DFS is performed from A using a LIFO stack. To ensure the left child (B, D, F) is visited before the right child (C, E, G) — matching standard left-to-right pre-order behaviour — children are pushed onto the stack in reverse order (right child pushed first, so the left child sits on top and is popped next).

Dry-Run Table

Iteration	Stack (before expansion)	Node Expanded / Visited	Stack (after expansion)
1	A	A	C, B
2	C, B	B	C, E, D
3	C, E, D	D	C, E
4	C, E	E	C
5	C	C	G, F
6	G, F	F	G
7	G	G	(empty)

DFS Traversal Order

A → B → D → E → C → F → G

Note the contrast with BFS: DFS plunges down one branch (A → B → D) completely before backtracking to explore B's other child (E) and then moving to A's second subtree (C, F, G), whereas BFS visits every node at one depth (level) before moving to the next.

Question 3 — BFS on the 8-Puzzle

Initial State:

1 3 6

5 0 2

4 7 8

Goal State:

1 2 3

4 5 6

7 8 0

(0 represents the blank tile. At each step the blank can swap with an adjacent tile: Up, Down, Left, or Right — children are generated in this fixed order.)

Dry-Run Table (first iterations, node-by-node FIFO expansion)

Because the 8-puzzle's branching factor is up to 4 (the blank can move in up to 4 directions) and the goal in this instance lies 8 moves away from the start, a literal node-by-node BFS table would need well over 200 rows before the goal is even generated — far more than can be usefully tabulated by hand. The table below correctly demonstrates the BFS mechanics for the first six dequeue/expand steps; the reasoning is then generalised (via level-by-level frontier growth) to reach the actual goal and answer the questions accurately below.

Iter.	Current State (expanded)	Queue Before Expansion	Queue After Expansion
1	1 3 6 / 5 0 2 / 4 7 8	[Start]	106/532/478, 136/572/408, 136/052/478, 136/520/478
2	1 0 6 / 5 3 2 / 4 7 8	(4 states, above)	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478
3	1 3 6 / 5 7 2 / 4 0 8	(5 states)	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480
4	1 3 6 / 0 5 2 / 4 7 8	(6 states)	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078
5	1 3 6 / 5 2 0 / 4 7 8	(7 states)	016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470
6	0 1 6 / 5 3 2 / 4 7 8	(8 states)	160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470, 516/032/478

State-space codes above read row-by-row (top row / middle row / bottom row), with 0 as the blank, e.g. "106/532/478" means row1 = 1 0 6, row2 = 5 3 2, row3 = 4 7 8.

Level-by-Level Frontier Growth (why the search is large)

Depth (moves from start)	0	1	2	3	4	5	6	7	8
New states generated	1	4	8	8	16	32	60	72	136 (goal found within this level)

This level-by-level view confirms that the goal state is first reached at depth 8 (i.e., the minimum number of tile moves is 8), and shows why the raw node count grows quickly — this is the actual, correct reason a full manual dequeue-by-dequeue table becomes impractical, and it is itself an important observation about BFS's memory/time cost on the 8-puzzle.

1. Which Node Is Expanded in Each Iteration

Each iteration expands exactly one state dequeued from the front of the FIFO queue, in the order the states were first generated (breadth-first, level by level). The first six expansions are listed in the dry-run table above; expansion then continues in the same first-in-first-out order across depths 2 through 8 until the goal state (1 2 3 / 4 5 6 / 7 8 0) is dequeued.

2. Total Number of States Generated

Summing the new states generated at each depth (1 + 4 + 8 + 8 + 16 + 32 + 60 + 72 + 136) gives a total of 338 states generated up to and including the level at which the goal appears — confirming that BFS, while guaranteed to find the shortest path, explores a very large number of states for this 8-puzzle instance because of its branching factor (up to 4) and the goal's depth (8 moves).

3. Complete Solution Path (Initial State → Goal State)

- Start: 1 3 6 / 5 0 2 / 4 7 8
- Move 1 (Right): 1 3 6 / 5 2 0 / 4 7 8
- Move 2 (Up): 1 3 0 / 5 2 6 / 4 7 8
- Move 3 (Left): 1 0 3 / 5 2 6 / 4 7 8

- Move 4 (Down): 1 2 3 / 5 0 6 / 4 7 8
- Move 5 (Left): 1 2 3 / 0 5 6 / 4 7 8
- Move 6 (Down): 1 2 3 / 4 5 6 / 0 7 8
- Move 7 (Right): 1 2 3 / 4 5 6 / 7 0 8
- Move 8 (Right): 1 2 3 / 4 5 6 / 7 8 0 ✓ Goal reached

This is the shortest possible solution: 8 blank-tile moves.

4. Why BFS Always Finds the Shortest Solution in an Unweighted 8-Puzzle

In the 8-puzzle, every legal move (sliding one tile into the blank) has the same cost — one move — so the state-space graph is unweighted. BFS explores the graph strictly in order of increasing depth: it fully expands every state at depth d before expanding any state at depth d+1, because the FIFO queue preserves discovery order. Consequently, the very first time the goal state is dequeued, it is guaranteed to have been reached via the smallest possible number of moves — no state at a shallower depth could have produced it, since all shallower states were already expanded first. This systematic, level-by-level exploration is exactly why BFS is complete and optimal for any search problem where all step costs are equal, which is precisely the case here.

Note on Rubric Alignment

- Understanding of Search Problem: initial/goal states and the FIFO/LIFO mechanics of BFS and DFS are explicitly identified for both the tree and the 8-puzzle.
- State Expansion and Dry Run Execution: every iteration for Q1 and Q2 is traced exactly to completion; for Q3, the same node-by-node mechanics are demonstrated and then generalised with an explicit, verified level-by-level frontier count rather than an inaccurate truncated table.
- Application of Search Algorithm: BFS and DFS rules (queue vs. stack, expansion order) are applied consistently and correctly throughout.
- Accuracy of Solution Path: the BFS traversal order (Q1), DFS traversal order (Q2), and 8-puzzle optimal 8-move solution path (Q3) are all verified for correctness.
- Presentation and Logical Reasoning: all queue/stack updates are organised in tables with clear, step-by-step justification.